SYMPRAXIS CONSULTING

"Auth Is Easy"

A 101 Guide to Authentication & Authorization in Microsoft 365 Development

Julie Turner

SYMPRAXIS CONSULTING



julieturner.net/me

Julie Turner

Partner / CTO · Sympraxis Consulting

- Working with SharePoint since **2007**
- **Microsoft MVP**, Office Apps & Services since 2017
- M365 & Power Platform **Community Team** since 2019
- Open-source co-maintainer: **PnPjs** & **hTWOo**
- Co-host: ***Ask Sympraxis***

Agenda

1. Auth is Easy”
2. Authentication vs. Authorization
3. The Mental Model: Tokens
4. OAuth Flows: Which One and When
5. SPFx & the Bearer Token
6. Your Own API: Two OBO Paths
7. Managed Identity: The Destination
8. Scenario Map: Given X, Use Y

01

"Auth Is Easy" 😅

“Auth is Easy” – the joke we all know

- Wrong audience on the token
- "Access denied" with zero context
- Worse – random message with zero context that turns out to be an auth issue.
- Three tabs deep in docs and your deadline doesn't care – OR AI is doing it for you.
- Expiring certificates and passwords

Why M365 auth gets complicated

- Choosing the right auth pattern
- Requesting the right permissions
- Managing tokens across API calls
- Diagnosing authorization failures

What this session is

- Understand the patterns that exist and why
- Know which pattern fits which scenario
- Make conscious tradeoff decisions instead of accidental ones

02

Authentication vs. Authorization

Authentication vs. Authorization

🔑 Authentication

- Proving an identity
- Which user or application is making the request?
- A person can sign in
- An application can prove its own identity

🚪 Authorization

- Checking access
- Can this identity perform this action on this resource?
- Permissions and resource rules apply
- A valid identity alone does not grant access

Delegated and application permissions

🤦 Authentication

- App acts on behalf of a signed-in user
- Access needs both the app's granted permissions and the user's access
- Example: Julie opens an app that reads her mail
- Common in SPFx, web apps and Teams tabs

🖥️ Application permissions (app-only)

- App acts as itself, without a signed-in user
- Access follows the app's granted permissions and resource restrictions
- Example: a scheduled service processes authorized mailboxes
- Common in background jobs and scheduled functions

Consent: who approves access?

- Delegated permissions: user consent where allowed, or admin consent
- Application permissions: admin consent required

Admin consent does not make delegated access app-only.

03

The Mental Model: Tokens

Access tokens: credentials for API requests

- Audience: a specific API
- Context: delegated or app-only access
- Lifetime: expires after a limited time
- API checks: token validity and permission to act

```
{
    "aud": "https://graph.microsoft.com", //audience
    "iss": "https://sts.windows.net/659b0b00-606f-47cb-8f49-eb12cf2d5663/", // issuer
    "iat": 1793806410, // issued
    "nbf": 1793806410, // don't use before
    "exp": 1793810858, // expires
    "app_displayname": "SharePoint Online Web Client Extensibility",
    "appid": "99999999-6177-487e-b8b5-cf950c1e598c",
    "family_name": "Turner",
    "given_name": "Julie",
    "idtyp": "user",
    "ipaddr": "999.999.999.999",
    "name": "Julie Turner",
    "oid": "99999999-1203-438e-ad69-1fb8b99159b4",
    "scp": "Acronym.Read.All AppCatalog.Read.All AppCatalog.Submit Bookmark.Read.All Calendars.Read
    Channel.Create Channel.ReadBasic.All ChannelMessage.Read.All Chat.Read Chat.ReadWrite ChatMember.Read
    Contacts.Read EduAssignments.ReadBasic EduRoster.ReadBasic ExternalItem.Read.All Files.Read Files.Read.All
    Files.ReadWrite Files.ReadWrite.All FileStorageContainer.Selected Group.Read.All Group.ReadWrite.All
    Group-Conversation.ReadWrite.All GroupMember.Read.All InformationProtectionPolicy.Read Mail.Read
    MailboxSettings.Read MailboxSettings.Read.All OnlineMeetingTranscript.Read.All People.Read People.Read.All
    Presence.Read.All PrinterShare.ReadBasic.All PrintJob.Create PrintJob.ReadBasic SensitivityLabel.Read
    Sites.FullControl.All Sites.Read.All Sites.ReadWrite.All Tasks.ReadWrite Team.ReadBasic.All
    TeamMember.Read.All TeamsAppInstallation.ReadWriteForTeam TeamsAppInstallation.ReadWriteSelfForTeam
    TeamsTab.Create TermStore.ReadWrite.All User.Read User.Read.All User.ReadBasic.All profile openid email",
    "tid": "999999999-606f-47cb-8f49-eb12cf2d5663",
    "unique_name": "julie@somedomain.com",
    "upn": "julie@somedomain.com",
}
```

04

OAuth Flows: Which One and When

OAuth flows

- Authorization Code: interactive user sign-in, most common in M365 dev
- Client Credentials: app-only, no user; background jobs, daemons, scheduled tasks
- On-Behalf-Of (OBO): user calls your API, your API acts downstream on their behalf
- Device Code: headless/CLI scenarios; many CLI frameworks offer this as an option
- *Deprecated: Implicit and ROPC (Resource Owner Password Credentials)*

Which flow fits which persona?

Flow	Persona	When to use it
Authorization Code	Delegated (user)	Web apps, interactive sign-in
Client Credentials	App-only	Background jobs, no user present
On-Behalf-Of (OBO)	Delegated → Delegated/App*	Your API calling Graph on user's behalf
Device Code	Delegated (user)	CLI tools, headless environments

05

SPFx & the Bearer Token

SPFx: what's built in

- SPFx includes auth under the hood — MSAL is there, you don't configure it – ***or use PnPjs!!!***
- The current user's (delegated) bearer token is available via context
- SharePoint REST APIs: token is right there — just use it, this is the happy path
- No extra app registration, no token acquisition code, no MSAL imports

SPFx shares Graph permission - grants across solutions

- Shared identity: SPFx solutions use a shared application identity to request Graph tokens.
- Shared grants: A delegated permission approved for one solution is available to other solutions using that identity.
- User boundary: Calls still run on behalf of the signed-in user and remain limited by that user's access.

The organizational decision

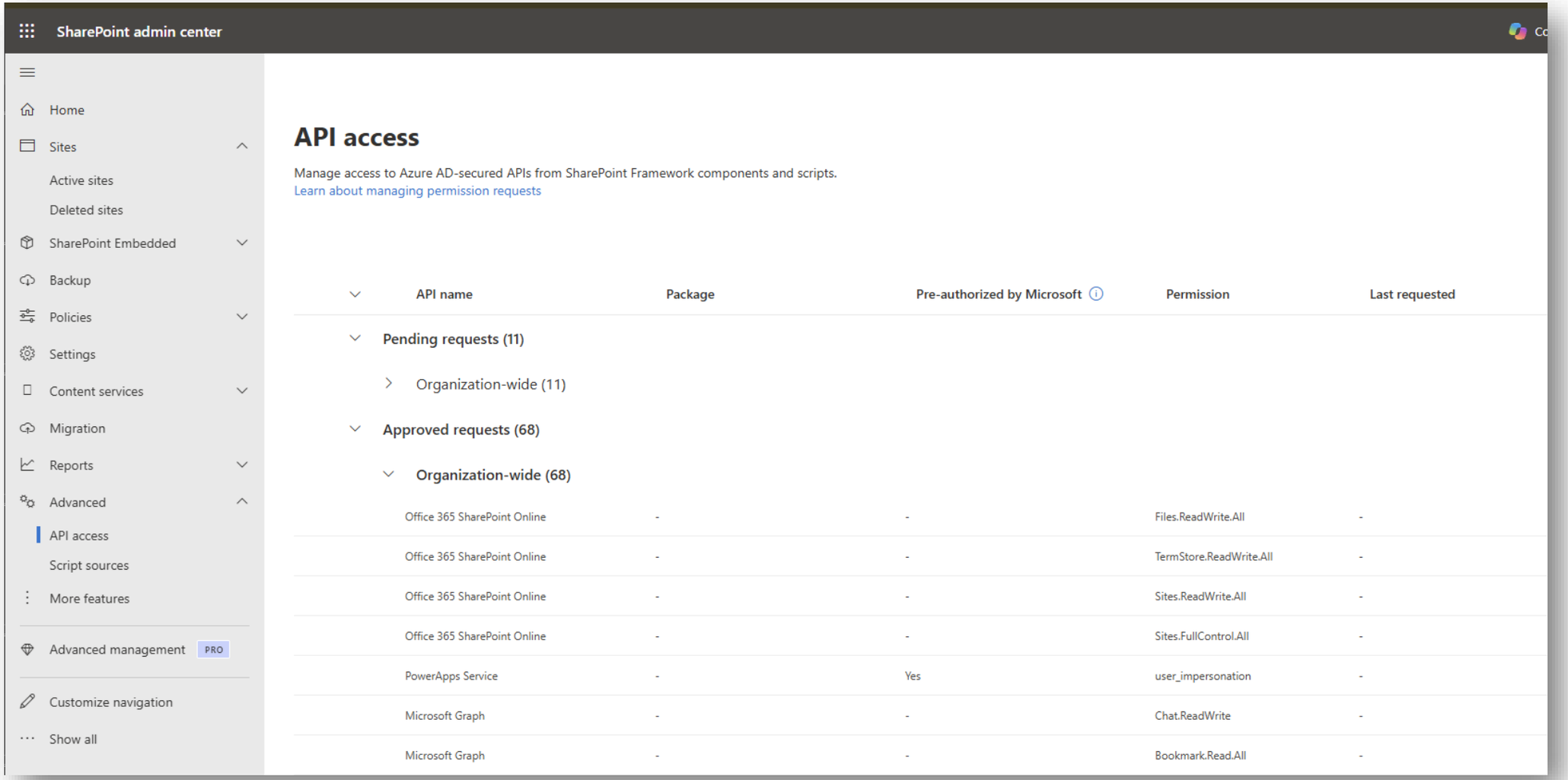
Accept the shared model

- Simpler: no extra infrastructure
- Fine for: low-sensitivity data, internal tooling, read-only org data
- Org consciously accepts the scope bleed tradeoff
- Most orgs with low-sensitivity M365 data choose this path

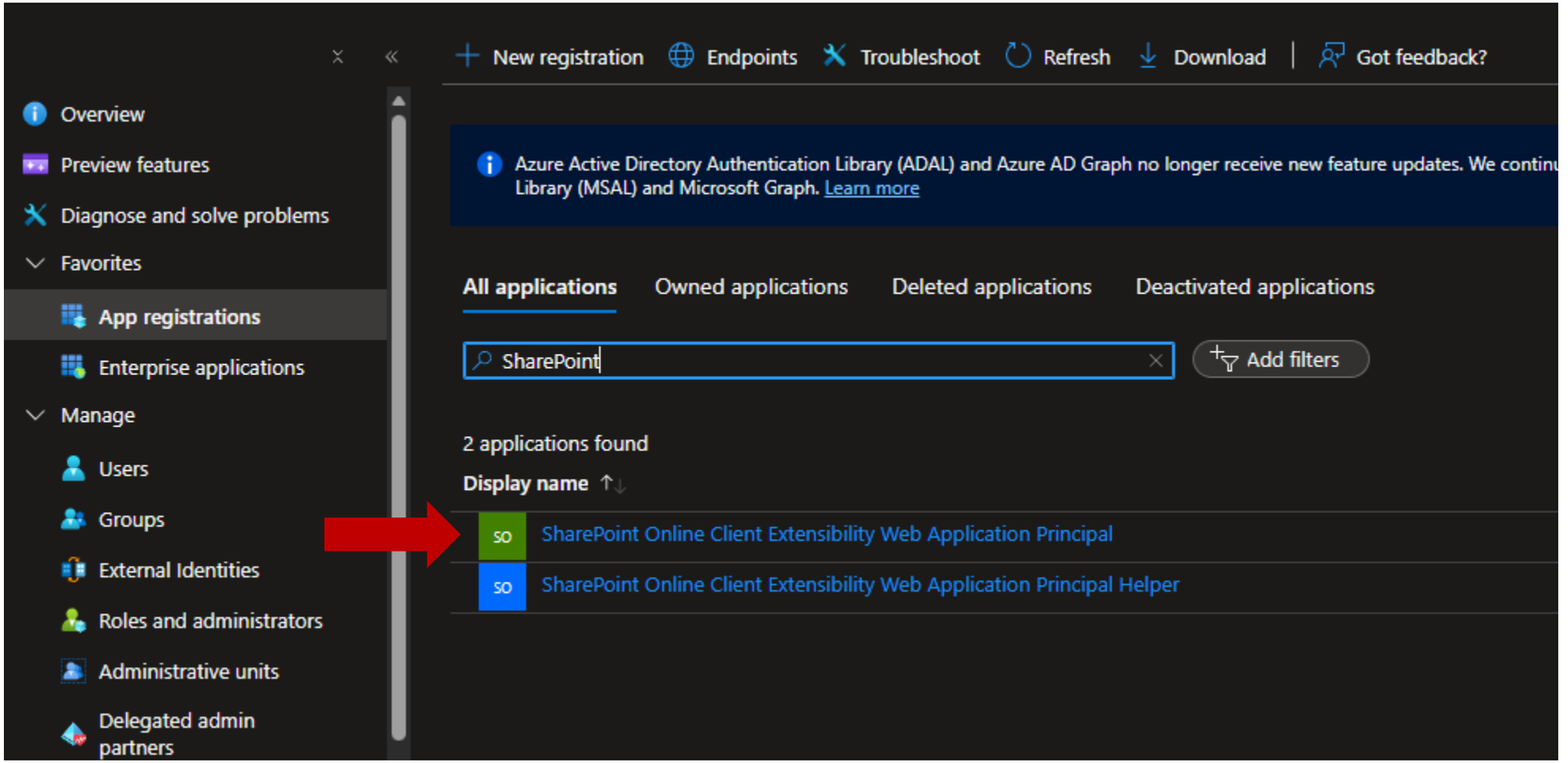
Build your own API back end

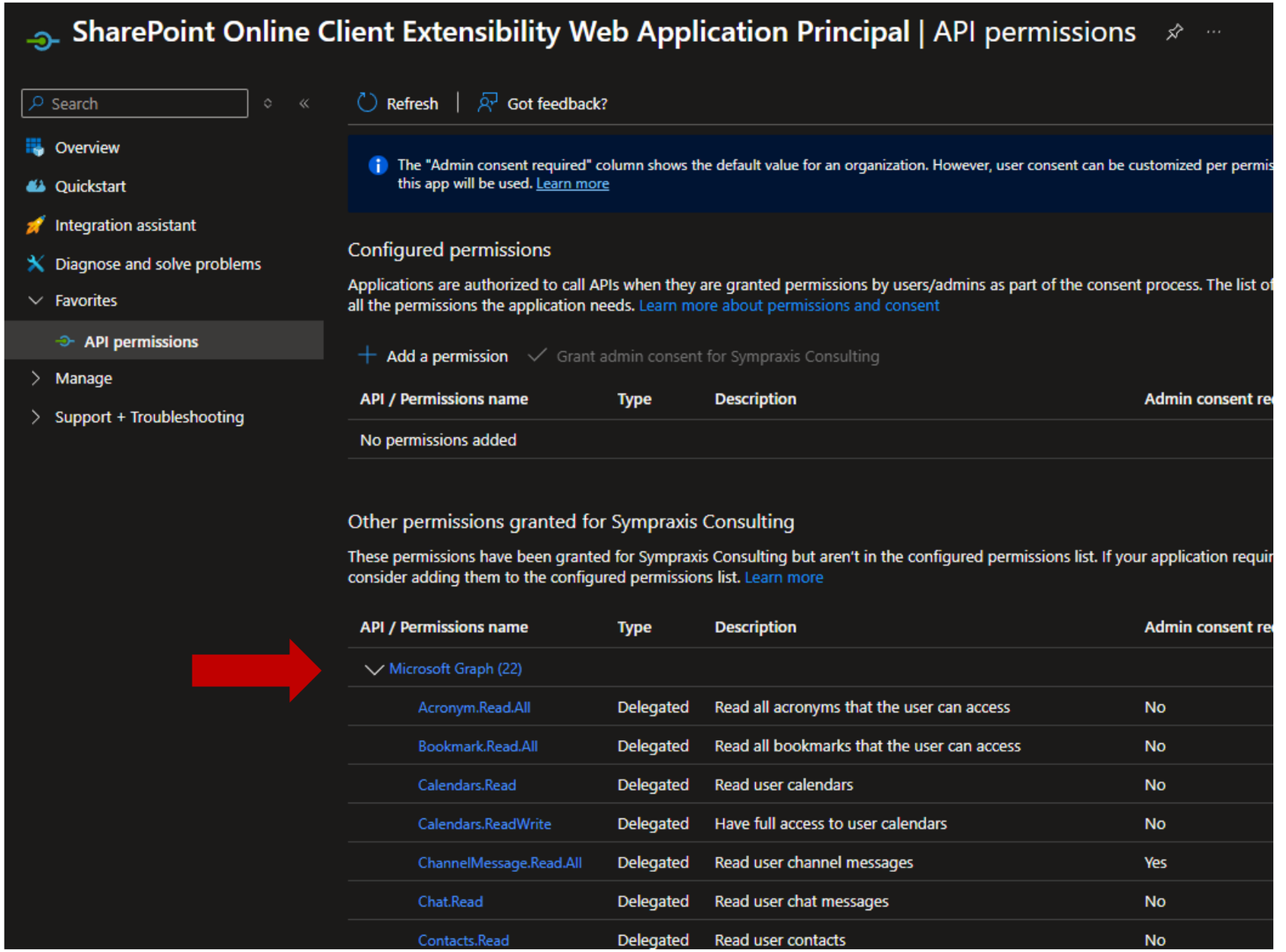
- Your SPFx solution calls your API
- Your API calls Graph with only the scopes it actually needs
- Real isolation: other SPFx solutions don't inherit your scopes
- Important for: sensitive data, mail, PII, HR data

SharePoint Admin Center – API Access



Shared Application Identity




SharePoint Online Client Extensibility Web Application Principal | API permissions
Search
Overview
Quickstart
Integration assistant
Diagnose and solve problems
Favorites
API permissions
Manage
Support + Troubleshooting
Refresh
Got feedback?
The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permis
this app will be used. Learn more
Configured permissions
Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of
all the permissions the application needs. Learn more about permissions and consent
Add a permission
Grant admin consent for Sympraxis Consulting
API / Permissions name
Type
Description
Admin consent re
No permissions added
Other permissions granted for Sympraxis Consulting
These permissions have been granted for Sympraxis Consulting but aren't in the configured permissions list. If your application requir
consider adding them to the configured permissions list. Learn more
API / Permissions name
Type
Description
Admin consent re
Microsoft Graph (22)
Acronym.Read.All
Delegated
Read all acronyms that the user can access
No
Bookmark.Read.All
Delegated
Read all bookmarks that the user can access
No
Calendars.Read
Delegated
Read user calendars
No
Calendars.ReadWrite
Delegated
Have full access to user calendars
No
ChannelMessage.Read.All
Delegated
Read user channel messages
Yes
Chat.Read
Delegated
Read user chat messages
No
Contacts.Read
Delegated
Read user contacts
No

06

Your Own API: Two OBO Paths

Path 1: App-only access

SPFx → Your API → Graph (app-only token)

- Your API validates the user token and authorizes the request
- Your API obtains a Graph token using its own identity
- Graph applies the app's permissions, not the user's

Path 2: Delegated access with OBO

SPFx → Your API → Graph (delegated token via OBO)

- Your API validates the user token and authorizes the request
- Your API uses OBO to obtain a delegated Graph token
- Graph access depends on delegated scopes and the user's permissions

Choosing between Path 1 and Path 2

Path 1: App-only access

- Downstream calls use the app's permissions
- Your API validates the users token and controls what the caller may do

Path 2: Delegated access with OBO

- Downstream calls preserve the user's identity
- Access depends on delegated scopes and the user's permissions

Token Validation

Token Validation Service

```
1   import jwt, { JwtPayload } from "jsonwebtoken";
2   import jwksClient from "jwks-rsa";
3   import { IAppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
4
5   const jwksSigningClient = jwksClient({
6     jwksUri: 'https://login.microsoftonline.com/common/discovery/keys',
7     requestHeaders: {},
8     timeout: 30000 // 30s (default)
9   });
10
11  const EXPECTED_TOKEN_ERRORS = ['TokenExpiredError', 'JsonWebTokenError', 'NotBeforeError', 'SigningKeyNotFoundError'];
12
13  export interface ITokenValidateService {···
16  }
17
18  export interface EntraJwtPayload extends JwtPayload {···
23  }
24
25  export class TokenValidateService implements ITokenValidateService {
26    private LOG_SOURCE = "TokenValidateService";
27    private _apu: IAppInsightUtil;
28    private _verifiedToken: EntraJwtPayload = null;
29    private _tenantId: string;
30    private _user: string;
31
32    public constructor(apu: IAppInsightUtil) {
33      this._apu = apu;
34    }
35
36    public get User() {
37      return this._user;
38    }
39
40    public async Validate(authorizationHeader: string): Promise<boolean> {···
165   }
166
167   private async getSigningKey(kid?: string) {···
170   }
171
172 }
```

Token Validation Service

```
40  public async Validate(authorizationHeader: string): Promise<boolean> {
41    let retVal = false;
42    try {
43      if (process.env.DEBUG == 'true' && !process.env.WEBSITE_SITE_NAME) {…
46      }
47      if (!authorizationHeader || authorizationHeader.length < 1) {…
58      }
59      if (!process.env.ApiAudience) {…
67      }
68      const accessToken = authorizationHeader.replace('Bearer ', '');
69      const decoded = jwt.decode(accessToken, { complete: true });
70      if (!decoded) {…
78      }
79
80      const key = await this.getSigningKey(decoded.header.kid);
        this._verifiedToken = jwt.verify(accessToken, key, {
82        algorithms: ['RS256'],
83        audience: process.env.ApiAudience
84      }) as EntraJwtPayload;
85      this._tenantId = this._verifiedToken.tid;
86      this._apu.Log(MessageType.Trace, {…
97      });
        if (this._tenantId.toLowerCase() == process.env.TenantId.toLowerCase()) {
99        // idtyp is only emitted on v2.0-style tokens; a client-credentials (app-only) token
100       // v1.0 endpoint carries no idtyp AND none of upn/unique_name/name
101       const hasNoUserClaims = !this._verifiedToken.upn && !this._verifiedToken.unique_name && !this._verifiedToken.name;
102       const isAppOnly = this._verifiedToken.idtyp === 'app' || hasNoUserClaims;
103       if (isAppOnly) {
104         const requiredRole = 'Api.Access';
105         if (!this._verifiedToken.roles?.includes(requiredRole)) {
106           this._apu.Log(MessageType.Trace, {…
115           });
116           return false;
117           // log + return false — authenticated, but not authorized for this API
118         }
119         this._user = process.env.SERVICE_ACCOUNT_UPN.toLowerCase();
120         retVal = true;
121       } else {
122         const claimSource = this._verifiedToken.upn ? "upn" : this._verifiedToken.unique_name ? "unique_name" : this._verifiedToken.name ? "name" : "(none)";
          this._user = this._verifiedToken.upn?.toLowerCase() || this._verifiedToken.unique_name?.toLowerCase() || this._verifiedToken.name?.toLowerCase();
          this._apu.Log(MessageType.Trace, {…
133       });
134       retVal = true;
135     }
136   } else {…
146   }
147 } catch (err) {
148   if (EXPECTED_TOKEN_ERRORS.includes(err?.name)) {…
155   } else {…
162   }
163 }
164 return retVal;
165 }
```

OBO Flow

OBO Token Service

```
import { ManagedIdentityCredential } from "@azure/identity";
import { AuthError, ConfidentialClientApplication, InteractionRequiredAuthError } from "@azure/msal-node";
import { IAppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";

const LOG_SOURCE = "OboTokenService";

// Downstream scopes are fixed server-side — clients never choose what the API can call on their behalf.
export const GraphScopes = ["https://graph.microsoft.com/.default"];
export const SharePointScopes = [`https://${process.env.Tenant}.sharepoint.com/.default`];

const miCredential = new ManagedIdentityCredential(
  process.env.ManagedIdentityClientId
    ? { clientId: process.env.ManagedIdentityClientId }  // user-assigned MI
    : undefined                                          // system-assigned MI
);

const getManagedIdentityAssertion = async (): Promise<string> => {
  const token = await miCredential.getToken("api://AzureADTokenExchange/.default");
  return token.token;
};

// Module-level ConfidentialClientApplication — created ONCE per Function host process so MSAL's
// in-memory token cache survives across invocations.
// Scale-out: each Function instance has its own cache, which is fine at modest scale.
// Larger implementations should consider distributed cache (Redis) via msal-node's DistributedCachePlugin.
const cca = new ConfidentialClientApplication({
  auth: {
    clientId: process.env.TokenClientId,
    authority: `https://login.microsoftonline.com/${process.env.TenantId}`,
    ...(process.env.TokenSecret
      ? { clientSecret: process.env.TokenSecret }              // local development only
      : { clientAssertion: getManagedIdentityAssertion })      // Azure: managed identity via FIC
  }
});
```

OBO Token Service

```
50  /**
51   * Exchanges the caller's bearer token (the same token validated by TokenValidateService) for a
52   * downstream access token using the On-Behalf-Of flow.
53   */
54  export async function getOboToken(authorizationHeader: string, scopes: string[], apu: IAppInsightUtil): Promise<string> {
55    const oboAssertion = authorizationHeader?.replace("Bearer ", "");
56    if (!oboAssertion) {
57      throw new OboError("Authorization header not provided", 401, "missing_assertion");
58    }
59
60    try {
61      const result = await cca.acquireTokenOnBehalfOf({ oboAssertion, scopes });
62      if (!result?.accessToken) {···
64      }
65      apu.Log(MessageType.Trace, {···
74      });
75      return result.accessToken;
76    } catch (err) {
77      if (err instanceof OboError) {
78        throw err;
79      }
80      apu.Log(MessageType.Exception, {···
85      });
86      if (err instanceof InteractionRequiredAuthError) {
87        // Consent required or MFA/conditional access step-up — the client must re-acquire its token.
88        throw new OboError("User interaction required", 403, err.errorCode, err.claims);
89      }
90      if (err instanceof AuthError) {
91        // Invalid, expired, or wrong-audience assertion.
92        throw new OboError("Token exchange failed", 401, err.errorCode);
93      }
94      throw err;
95    }
96  }
```

HTTP Endpoint

```
6   // Example API endpoint using OBO: the caller's token is validated, exchanged server-side for a Graph
7   // token (cached by MSAL), Graph is called as the user, and only the data is returned — never the token.
8   export async function getMyProfile(request: HttpRequest, context: InvocationContext): Promise<HttpResponseInit> {
9     const LOG_SOURCE = "getMyProfile";
10
11    const _apu = new AppInsightUtil();
12    _apu.Init(context.invocationId, context.traceContext?.traceParent ?? '');
13
14  >   _apu.Log(MessageType.Event, {···
21    });
22
23    try {
24      const authorizationHeader = request.headers.get("authorization");
25      const tokenValidateService = new TokenValidateService(_apu);
26      const validToken = await tokenValidateService.Validate(authorizationHeader);
27  >   if (!validToken) {···
38      }
39      _apu.AddCommonProperty("authUser", tokenValidateService.User);
40
41      // The validated bearer token IS the OBO assertion — no separate token in the request body.
42      const graphToken = await getOboToken(authorizationHeader, GraphScopes, _apu);
43
44      const graphResponse = await fetch("https://graph.microsoft.com/v1.0/me?$select=displayName,mail,jobTitle,department,officeLocation", {
45        headers: { Authorization: `Bearer ${graphToken}` }
46      });
47  >   if (!graphResponse.ok) {···
67      }
68
69      return {
70        status: 200,
71        jsonBody: await graphResponse.json()
72      };
73  >   } catch (err) {···
116   }
117 }
```

07

Managed Identity: The Destination for API Layer

Why Managed Identity is the pattern to aim for

- Most secure path: no credentials in code, no secrets to rotate, no app reg secrets to leak
- Your Azure Function (or other resource) gets an identity: Azure handles it
- Grant that identity the access it needs, MSAL is abstracted away entirely
- DefaultAzureCredential: picks up Managed Identity automatically in Azure

Local dev: the one caveat

☁️ In Azure

- Managed identity
- No app secrets to manage
- Permissions granted to the Azure identity

👨‍💻 Local development

- No local managed identity
- Dev app + certificate for app-only testing
- Permissions granted to the dev identity

08

Scenario Map: Given X, Use Y

Given X, use Y

Scenario	Pattern	Notes
SPFx → SharePoint REST	Bearer token from context	Built in, just use it
SPFx → Graph (low sensitivity)	Shared tenant app reg + delegated	Conscious org decision required
SPFx → Graph (sensitive / least privilege)	SPFx → your API → OBO → Graph	Path 1 or Path 2 depending on needs
Web app (no SPFx) → Graph	Authorization Code flow	Standard OAuth, MSAL handles it
CLI tooling	Device Code flow	Most CLI frameworks offer this
Teams tab → Graph	MSAL.js + NAA	Supported tab hosts; delegated Graph access
Office Add-in → Graph	MSAL.js + NAA	Supported Office hosts; delegated Graph access
Azure Function / Graph webhooks	Client Credentials + Managed Identity	App-only — Managed Identity is the answer
Local dev / Azure	DefaultAzureCredential chain	Cert, env vars, or dev tooling

Teams tabs and Office Add-ins: Graph access

NAA: delegated permissions for your app

- NAA (nested app authentication): MSAL.js requests Graph tokens for supported Teams tabs and Office Add-ins
- App identity: your Entra client ID; requested scopes in MSAL (e.g. Mail.Read)
- Consent: user or admin approval; access remains limited by the user
- Traditional SSO: token for your API, then backend OBO for Graph

Two things to remember

Know what tradeoffs you're accepting — and design toward Managed Identity.

Resources

- Access tokens in the Microsoft identity platform
- Manage access to Microsoft Entra ID-secured APIs
- Azure Identity client library for JavaScript
- Enable single sign-on in an Office Add-in with nested app authentication